

T1.4.B — Fix P0 Auth Dashboard

Validación de password en NextAuth · cierra agujero crítico

Owner: Celestino · 2026-05-02 · Branch `pr/t1.4.b-fix-auth-dashboard` · Commit `2f8e43f`

Status: Branch pusheado a GitHub. Build pasa. Determinismo del helper validado manualmente. **NO mergeado a main.** Owner configura `DASHBOARD_PASSWORD_SECRET` en Vercel y deriva el password para usuarios existentes ANTES de mergear.

Bug que se cerró: En `landing/auth.ts` línea 13–48, la función `authorize` recibía password pero NUNCA lo validaba. Cualquiera con un email de subscriber activo podía entrar al dashboard y cancelar la suscripción. Identificado en T1.4.A.

1. Archivos modificados / creados

Archivo	Δ líneas	Tipo	Rol
<code>landing/lib/dashboard-auth.ts</code>	+58 / -0	A	deriveDashboardPassword + constantTimeEqual
<code>landing/auth.ts</code>	+15 / -1	M	Validación de password antes del lookup de subscription
<code>landing/scripts/derive-dashboard-password.ts</code>	+43 / -0	A	CLI helper para derivar password de un customer
<code>.env.example</code>	+23 / -0	M	Documenta <code>DASHBOARD_PASSWORD_SECRET</code>
TOTAL	+139 / -1	4 archivos	

2. Diff resumido

a) `landing/lib/dashboard-auth.ts` (nuevo)

```
export function deriveDashboardPassword(
  customerId: string,
): string | null {
  const secret = process.env.DASHBOARD_PASSWORD_SECRET?.trim();
  if (!secret || !customerId) return null;
  const digest = crypto
    .createHmac("sha256", secret)
    .update(customerId, "utf8")
    .digest("hex");
  return "dona-" + digest.slice(0, 12);
}

export function constantTimeEqual(a: string, b: string): boolean {
  if (a.length !== b.length) return false;
  const ab = Buffer.from(a, "utf8");
  const bb = Buffer.from(b, "utf8");
  return crypto.timingSafeEqual(ab, bb);
}
```

b) `landing/auth.ts` — validación insertada

```
// Lookup customer (sin cambios).
const customers = await getStripe().customers.list({email, limit: 1});
if (customers.data.length === 0) return null;
```

```
const customer = customers.data[0];

// T1.4.B – validar password ANTES de buscar la suscripción.
const expected = deriveDashboardPassword(customer.id);
if (!expected || !constantTimeEqual(password, expected)) {
  return null;
}

// Lookup subscription (sin cambios).
const subscriptions = await getStripe().subscriptions.list(...);
if (subscriptions.data.length === 0) return null;
```

c) landing/scripts/derive-dashboard-password.ts (nuevo)

CLI utility para que el owner derive el password de un usuario y se lo entregue por canal seguro. **No** se importa en el bundle de producción. Uso:

```
cd landing
DASHBOARD_PASSWORD_SECRET= \
  npx tsx scripts/derive-dashboard-password.ts cus_xxx

→ stdout: dona-fc90aa9944c7 (ejemplo, no real)
```

d) .env.example — sección nueva

23 líneas que documentan DASHBOARD_PASSWORD_SECRET, su propósito, cómo generarla, dónde configurarla (solo Vercel) y cómo derivar el password de un usuario existente.

3. Decisiones de diseño

Opción considerada	Decisión
Magic Link (email-only, sin password)	Rechazada · requiere infra de envío de mail + tokens single-use; cambio mucho más grande que e
Master password único compartido	Rechazada · filtración total con un solo email
Hash bcrypt en Stripe Customer metadata	Rechazada · primer login requiere flow 'create password'
Tabla nueva en backend con hashes	Rechazada · requiere endpoint nuevo + Alembic + flow set-password
Password derivado HMAC + customer_id ELEGIDA · sin storage, determinístico, único por usuario, reusa entropía del secret	

Por qué esta opción

- **Mínima:** sin DB nueva, sin migración, sin endpoint nuevo, sin Alembic.
- **Sin path permisivo:** si la env var falta, todos los logins fallan. Mejor bloquear que aceptar.
- **Resistente a timing oracles:** compareDigest sobre buffers UTF-8.
- **Validar antes del lookup de subscription:** no revela si el customer existe en Stripe (mismo error para password incorrecto y customer inexistente).
- **Sin password reset complicado:** el password es derivable, no se pierde. Si el usuario lo olvida, el owner lo regenera con el helper.

4. Comandos ejecutados y resultado

Comando	Resultado
npx tsc --noEmit	0 errores
npm run lint	16 problems · todos preexistentes (any types en otros archivos, img tag, etc). 0 nuevos por T1.4.B.
npm run build	✓ Compiled successfully · ✓ TypeScript pasa · 14 rutas registradas
Test manual del helper	Determinismo OK · sin secret → exit 2 · distinto secret/customer → distinto password

Test manual: outputs reales

```
$ DASHBOARD_PASSWORD_SECRET=test-secret-not-real-12345 \  
  npx tsx scripts/derive-dashboard-password.ts cus_TEST_001  
dona-fc90aa9944c7  
  
$ # mismo input → mismo password (determinismo)  
$ DASHBOARD_PASSWORD_SECRET=test-secret-not-real-12345 \  
  npx tsx scripts/derive-dashboard-password.ts cus_TEST_001  
dona-fc90aa9944c7  
  
$ # distinto customer → distinto password  
$ DASHBOARD_PASSWORD_SECRET=test-secret-not-real-12345 \  
  npx tsx scripts/derive-dashboard-password.ts cus_TEST_002  
dona-0bb38380f5d3  
  
$ # distinto secret → distinto password  
$ DASHBOARD_PASSWORD_SECRET=diferente-secret \  
  npx tsx scripts/derive-dashboard-password.ts cus_TEST_001  
dona-1386e4148cd6  
  
$ # sin secret → exit 2  
$ npx tsx scripts/derive-dashboard-password.ts cus_TEST_001
```

```
Error: DASHBOARD_PASSWORD_SECRET no configurada o customer_id vacío.  
exit=2
```

Nota: el secret 'test-secret-not-real-12345' es un valor ficticio usado solo para verificar la lógica. En Vercel se debe configurar uno de alta entropía generado con `secrets.token_urlsafe(32)`.

5. Criterios de aceptación

Criterio	Cómo se cumple
Password incorrecto → login rechazado	constantTimeEqual retorna false → authorize devuelve null → NextAuth muestra error ge
Email sin sub activa → login rechazado	subscriptions.list({status:'active'}) vacío → return null
Email + sub activa + password correcto → login OK	Path completo pasa, retorna user con stripeCustomerId, etc.
No filtrar secrets en logs ni errores	El secret nunca se loguea. El password derivado solo viaja por stdout del CLI helper loca
Build/lint/test pasa	tsc 0 errores, lint baseline igual, build OK, helper test OK
Reportar archivos / diff / riesgos / próximos	Este documento + resumen al final del turno

6. Riesgos y próximos pasos

Riesgos abiertos

Riesgo	Severidad	Mitigación
Si DASHBOARD_PASSWORD_SECRET se filtra, todos los passwords son reseteados	Alta	Reseteo de todos los passwords en Vercel; generar nuevos passwords para todos los usuarios
Sin rate limiting en /login (brute force de emails)	Media	Pendiente — agregar middleware o turnstile en T1.4.E o antes si se detecta ataque
Owner debe entregar el password al usuario por canal seguro	Crisis	Recomendado: por WhatsApp via Dona, no por email (Stripe customer email)
Si el usuario olvida el password, regenerarlo deja el login no operable hasta que se resetea	Baja	El password es determinístico — mismo customer_id + mismo secret = mismo password
NextAuth muestra mismo mensaje para todos los fallos de login	Baja (es feature)	Previene oracle de existencia de email/sub

Próximos pasos para el owner

Antes de mergear este PR:

- 1. **Generar DASHBOARD_PASSWORD_SECRET:** `python -c "import secrets; print(secrets.token_urlsafe(32))"`
- 2. **Configurarlo en Vercel** (Settings → Environment Variables, scope **Production**).
- 3. **Para cada usuario activo existente** (Premium/Pro): buscar su `cus_xxx` en Stripe Dashboard y derivar el password con el helper local. Anotar el password.
- 4. **Mergear el PR:** <https://github.com/celestinojbm/Dona-agent/compare/main...pr/t1.4.b-fix-auth-dashboard>
- 5. **Vercel redespiega** automáticamente. Inmediatamente después del redeploy, el login con el password viejo (o sin password) deja de funcionar.
- 6. **Entregar el password al usuario** por WhatsApp via Dona

Importante: entre el merge y la entrega del password, los usuarios existentes **no podrán acceder al dashboard**. Coordinar con el primer usuario activo (vos mismo) para minimizar ventana. Si se quiere ventana cero, derivar password ANTES del merge y comunicar via WhatsApp justo después del merge.

Próximos PRs en T1.4 (orden aprobado)

- T1.4.C:** Backend read-only POST /internal/usuario-resumen (HMAC). Sin riesgo (solo SELECTs).
- T1.4.D:** Landing /api/dashboard-data, proxy server-side al backend.
- T1.4.E:** Dashboard UI con datos reales — saldo, plan, historial, próxima renovación.
- T1.4.F:** Cancelación segura — `cancel_at_period_end` en vez de `cancel()` inmediato.